

CMPT 383: Vitamin #9

Anders Miltner
miltner@cs.sfu.ca

Introduction

This Vitamin is to help you practice lifetimes and borrowing in Rust. The test suite is provided in `src/lib.rs`. You should fill out the function definitions in `src/functions.rs`.

This submission will be partially autograded. There are some portions of the assignment that are ungraded, and some that will be graded. We provide a (partial) test suite for partial validation. You can run these tests by opening a terminal in the `v7` directory, and running `cargo test`.

We have omitted all imports. If you import additional functions, you may get a zero on the assignment.

1 prefixes

The `prefixes` function returns all prefixes of a provided string. The empty string is a prefix of every string. For example the prefixes of "Hi" are "", "H", and "Hi".

Using string slices will be quite helpful. As a reminder, you can get the substring of `s` between `i` (inclusive) and `j` (exclusive) with the syntax: `&s[i..j]`. For example: `&"Hello"[1..3] == "e1"`

2 return_if_satisfies_both

The `return_if_satisfies_both` function is provided a higher-order function (`f`) and two borrowed values (`x` and `y`). If `f(x)` and `f(y)` both evaluate to true, a `Some Option` should be returned, containing a tuple with both `x` and `y`. If at least one of them evaluate to false, a `None` option should be returned. Currently, the function signature returns an option with incorrect lifetimes: `Option<(&'static T, &'static T)>`. To get the desired function to compile, you will need to change the lifetime in the return type.

3 map

The `map` function will involve re-implementing the Haskell `map` function on the `List` data type. The `List` data type involves using a `Box`. You can see an example of building lists by looking at the tests. Apply the provided

4 concat

The `concat` function will involve re-implementing the Haskell `(++)` function on the `List` data type. Note: you are permitted to use the `clone()` method in exactly one of the cases.